Day 5

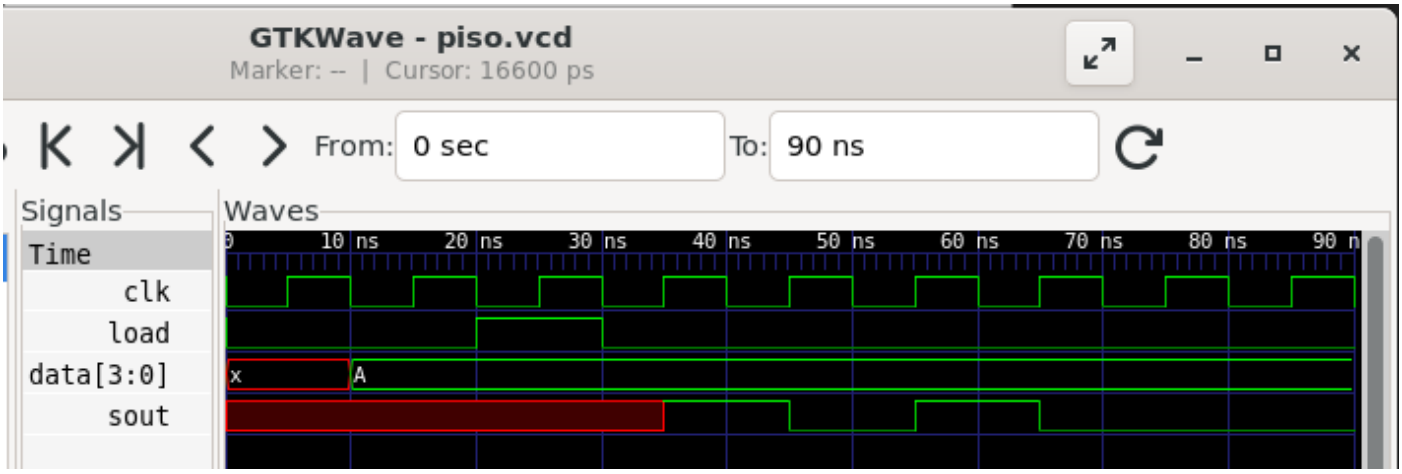
Parallel In Serial Out (PISO)

Verilog Code:

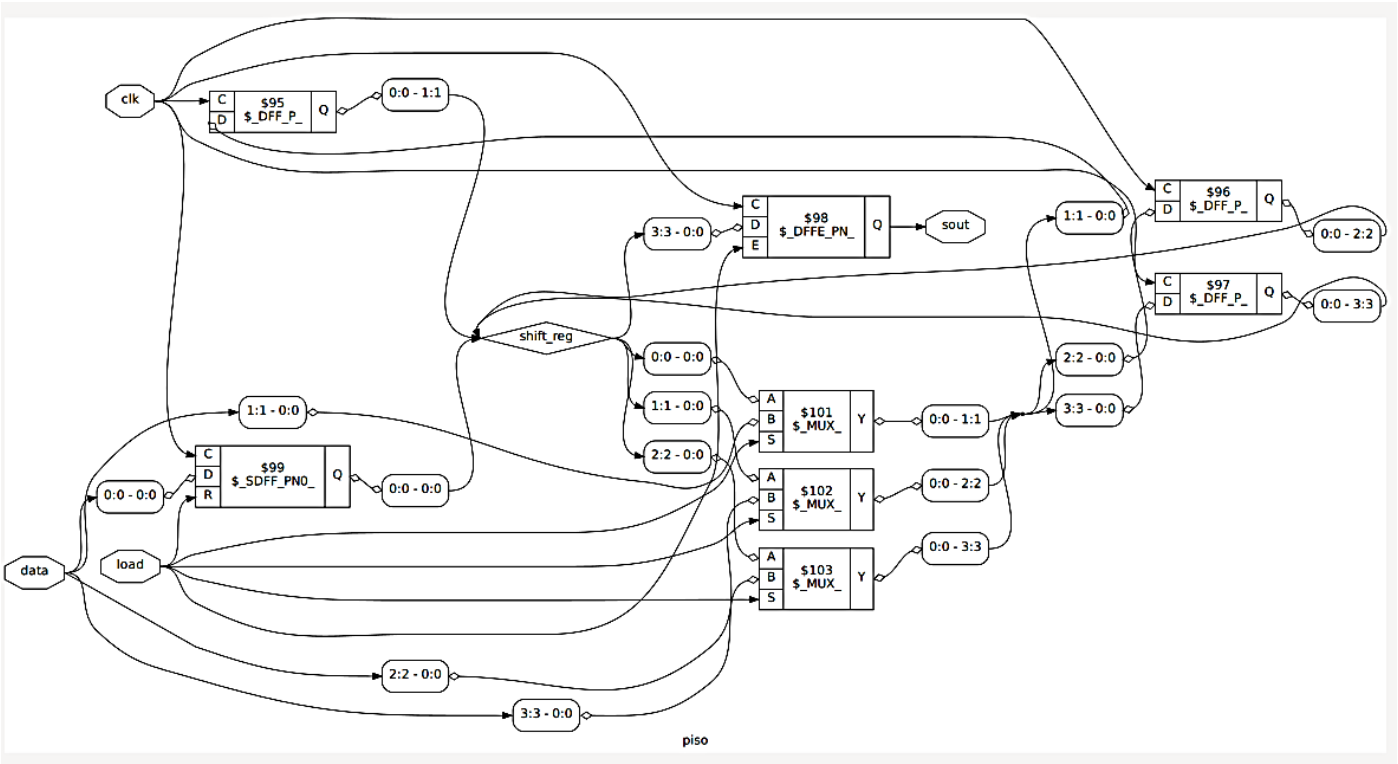
```
module piso(
   input clk, load,
   input [3:0] data,
   output reg sout
);
   reg [3:0] shift_reg;

   always @(posedge clk) begin
      if(load)
         shift_reg <= data;
      else begin
         sout     <= shift_reg[3];
         shift_reg <= shift_reg << 1;
      end
   end
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb;
reg clk, load;
reg [3:0] data;
wire sout;

piso uut(.clk(clk), .load(load), .data(data), .sout(sout));

initial begin
$dumpfile("piso.vcd");
$dumpvars(0, tb);
end
initial begin
 clk = 0;
 forever #5 clk = ~clk;
end
initial begin
 load = 1'b0; #10
 data = 4'b1010;#10
 load = 1'b1;#10
 load = 1'b0;#10
 #10
 #10
 #10
 #10
 #10
 $finish;
end
endmodule
```



Simulation and Synthesis output:



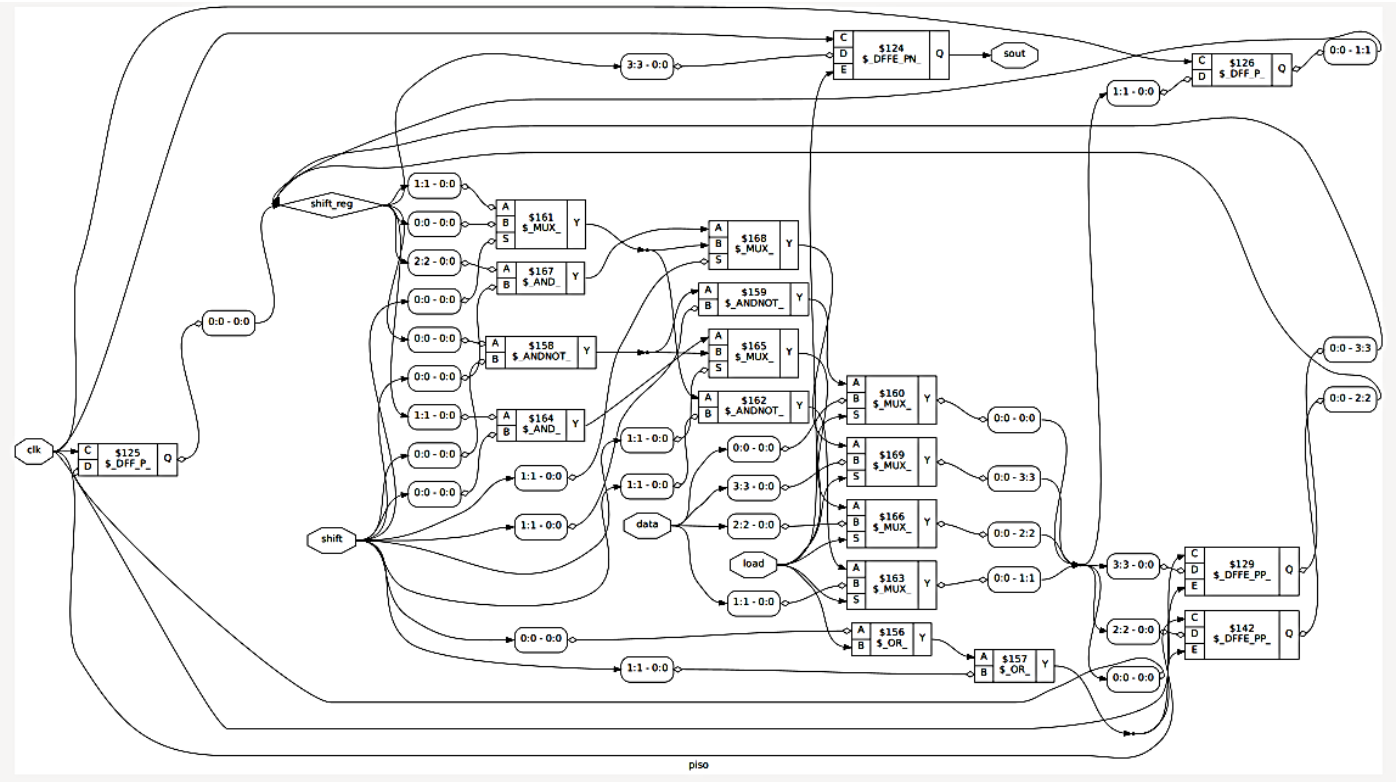
2.26. Printing statistics.

=== piso ===

```
   Number of wires:                 6
   Number of wire bits:            15
   Number of public wires:          5
   Number of public wire bits:     11
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 8
     $_DFFE_PN_                     1
     $_DFF_P_                       3
     $_MUX_                         3
     $_SDFF_PN0_                    1
```

PISO using a variable for shifting instead of a constant

```
module piso (
   input clk, load,
   input [1:0] shift,
   input [3:0] data,
   output reg sout
);
   reg [3:0] shift_reg;

   always @(posedge clk) begin
      if(load)
         shift_reg <= data;
      else begin
         sout      <= shift_reg[3];
         shift_reg <= shift_reg << shift;
      end
   end
endmodule
```



2.26. Printing statistics.

```
=== piso ===

   Number of wires:                 17
   Number of wire bits:             27
   Number of public wires:           6
   Number of public wire bits:      13
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                 19
     $_ANDNOT_                       3
     $_AND_                          2
     $_DFFE_PN_                      1
```

```
$_DFFE_PP_          2
$_DFF_P_            2
$_MUX_              7
$_OR_               2
```

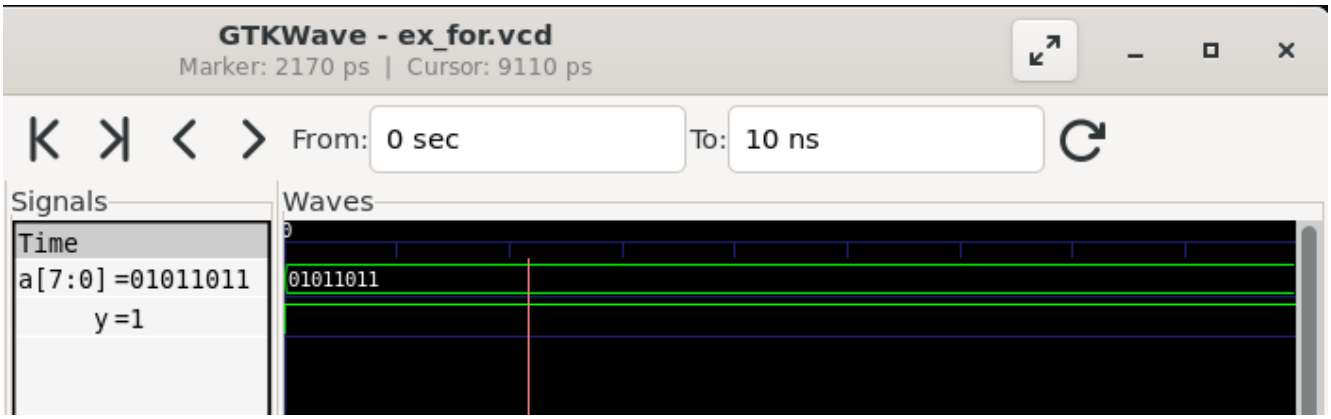
Note: We can see that the number of cells in the variable type shifting has increased drastically, the thing to learn here is, the parameter/ constants are compile time constant not a runtime signal, whereas a variable is a runtime constant which has become barrel shifter here.
The different D flip-flops here are $_DFF [E=enable] [S=sync reset or P=async] [polarity of clock P/N] [polarity of reset 0/1]

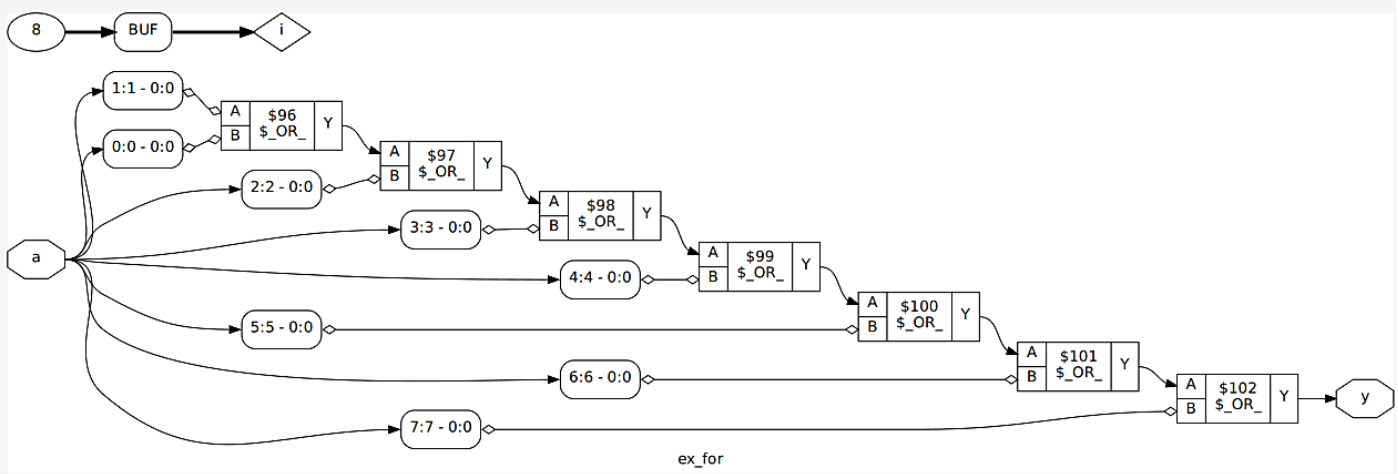
Example using for loop, or (|) reduction operation

```
module ex_for(
 input [7:0] a,
 output reg y);

 integer i;
 always @(*) begin
   y = 0;
   for(i=0; i<8; i=i+1)
     y = y | a[i];
  end
endmodule
```

Testbench:

```
`timescale 1ns/1ps
module tb;
reg [7:0] a;
wire y;
ex_for uut(.a(a), .y(y));

initial begin
$dumpfile("ex_for.vcd");
$dumpvars(0, tb);
end

initial begin
a = 8'b01011011; #10;
$finish;
end
endmodule
```





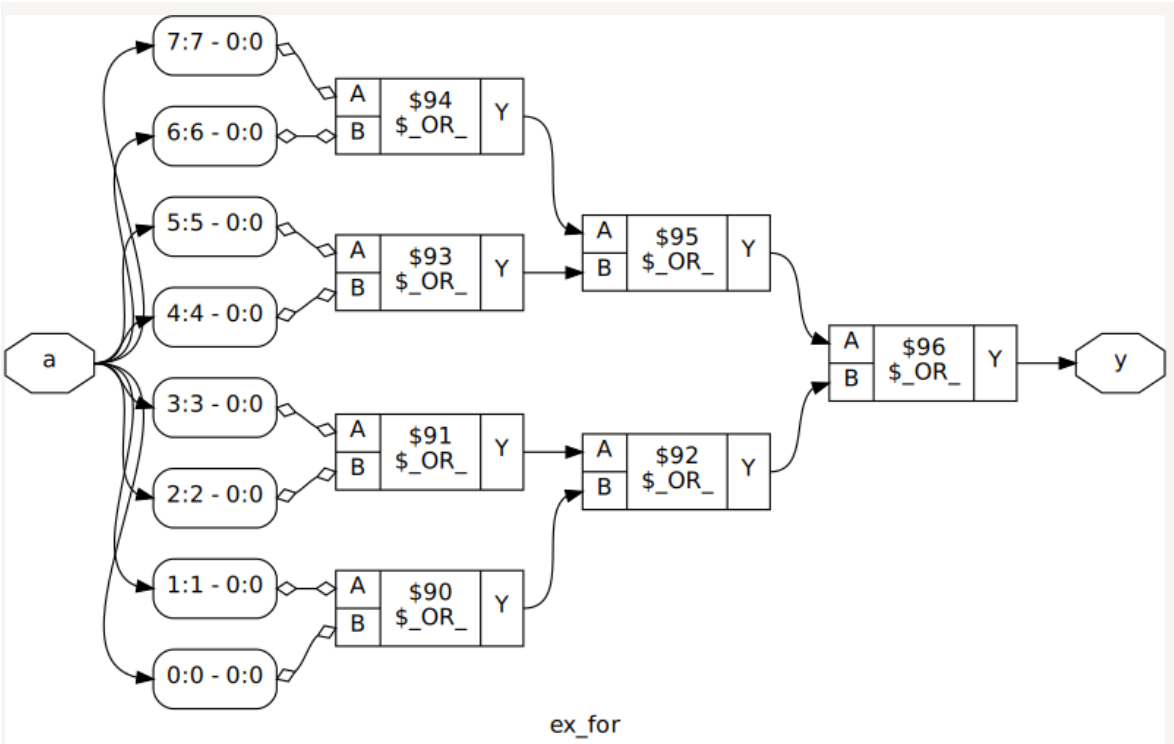
2.26. Printing statistics.

=== ex_for ===

Number of wires: 9
Number of wire bits: 47
Number of public wires: 3
Number of public wire bits: 41
Number of memories: 0
Number of memory bits: 0
Number of processes: 0
Number of cells: 7
$_OR_ 7

Same code using dataflow modelling without for loop

```
module ex_for(
 input [7:0] a,
 output y);

 assign y = |a;
endmodule
```



2.26. Printing statistics.

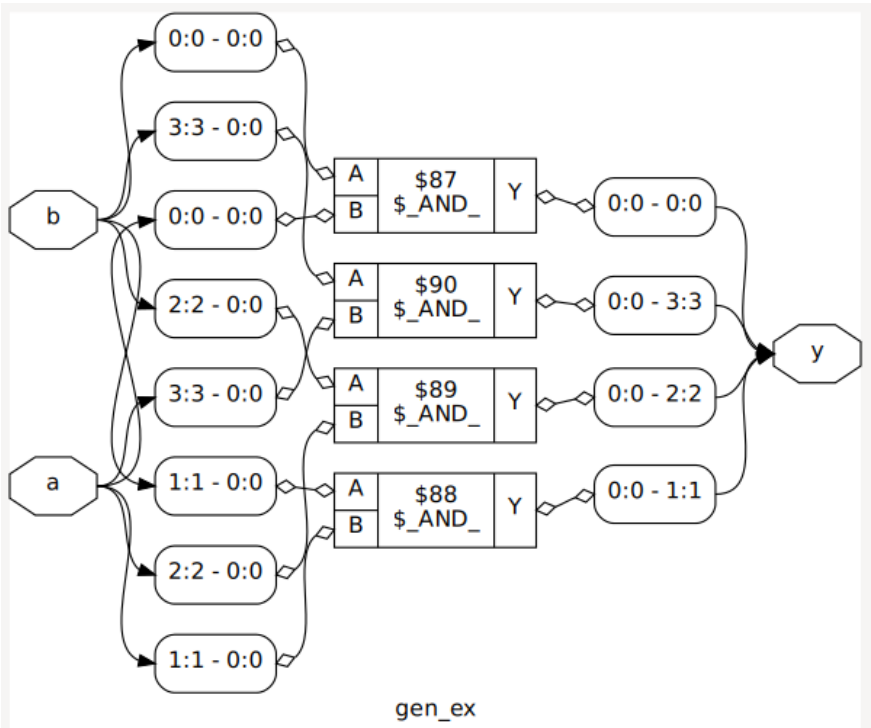
=== ex_for ===

```
   Number of wires:                 8
   Number of wire bits:            15
   Number of public wires:          2
   Number of public wire bits:      9
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 7
     $_OR_                          7
```

Important Note:
Same number of gates, same area, same stats — but completely different **timing**. The tree has a critical path of only 3 gates. The chain has a critical path of 7 gates. In a real chip running at high frequency, the chain version would fail timing and the tree version would pass.
We can see that; the synthesiser is not obligated to build the fastest structure from the code. How you write the loop affects the dependency chain it sees, which affects the shape of the logic tree it builds.

Using Generate

```
module gen_ex #(parameter N = 8)(
 input [N-1:0] a,b,
 output [N-1:0] y
 );

 genvar i;
 generate
   for(i=0; i<N; i=i+1) begin : gen_and
      assign y[i] = a[i] & b[i];
   end
 endgenerate
endmodule
```



2.26. Printing statistics.

```
=== gen_ex ===

   Number of wires:                 3
   Number of wire bits:            12
   Number of public wires:          3
   Number of public wire bits:     12
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 4
     $_AND_                         4
```

Generate is a compile time loop - it runs before synthesis even starts, during elaboration. It's not hardware itself, it's an instruction to the tool to *replicate* hardware N times.
A **genvar** is a special type of variable used **only** during the elaboration phase of synthesis to control generate loops.
Why not just use reg or integer?
In Verilog, reg and integer variables are intended for **procedural** logic (inside always blocks) where values can change while the chip is running.
Because a generate loop creates **fixed physical hardware**, the compiler needs a variable that disappears once the design is "built." That variable is the genvar.

Use cases of Generate:
Scalable datapaths: Building an N-bit structure without writing N lines
Conditional hardware: Including and excluding blocks based on the parameters
This is how real IPs ship with optional features — same RTL, different parameter = different silicon.
Memory arrays — replicating register banks